Six jobs, not one machine: obstruction, standing, transport, comparison, shadow, leftover (CODA-GTLA)

Subtitle: A leftover class is not standing

Author: Jeremy H. Carroll

Date: 2026-09-28

Version: 3.2.1 — preprint

License: Creative Commons Attribution 4.0 International

CODA Architecture & Layers

CODA stands for the Calculus of Obstruction, Descent, and Authority. It is an exact, finite mathematical apparatus that enforces strict claim discipline by separating linear repairs, transport certificates, and finite descent from ungrounded authority claims. The system comprises eight operational layers governed by a single overarching authority architecture (GTLA).

Layer	Abbreviation	Layer / Component Name & Scope
1	**FOC**	Finite Obstruction Calculus
2	**MUR**	Meta & Universal Re-Presentation (what presentations exist)
3	**CS**	Continuum Shadow (what continuum presentation may descend to a finite calculus system)
4	**CS_Q**	Continuum Shadow to Continuum Bridge (how continuum presentations descend)
5	**Q**	The Continuum (continuum presentations via completion and enclosure)
6	**A**	Anomalon (global residuals not removed and typed instead)
7	**OR**	Omniversal Re-Presentation (what presentations can transport)
8	**FDA**	Finite Descent Authority (what presentation may descend to evaluation and authorization)
9	**GTLA**	Layered Authority Architecture for CODA

Abstract

This note is a spine, not a new calculus. Six companion papers already do six different jobs: measuring an obstruction leftover, recording standing, transporting a picture, comparing overlaps, reading a shadow, and computing a leftover vector. A leftover class is not standing. The Lean in this package records presentations, spans, traces, and a firewall on declared finite records. Theorems that live in FOC, FDA, MUR, OR, CS, and A are cited there, not re-proved here. The package includes the vendored `Coda.A` residual algebra used by its wrappers; the other companion results remain external citations.

Introduction

It is easy to treat six desks as one machine. You measure a leftover, you move a picture, you glue two pairs, you look at a blurry photo, and you talk as if any one of those had answered the standing question. They have not.

A picture. Six jobs, six papers:

1. **Obstruction** (FOC). A defect is a vector you may change only by adding a licensed linear repair. What cannot be cancelled is a leftover number.
2. **Standing** (FDA). Standing is a recorded pair: a pipeline actually produced a terminal, and seven halt guards derived from evidence all hold. A proposal is not that pair.
3. **Transport** (MUR). A commuting square says: look after the move, or move after the look. Collisions travel. The square is not target standing.
4. **Comparison** (OR). Two overlapping pairs glue to a unique triple exactly when they agree on the shared slot. Glue is not a grant.
5. **Shadow** (CS). A fine look can be forgotten to a coarse one. Four Boolean pairs share two first coordinates; the first coordinate does not determine the second. A blurry photo is not the original.
6. **Leftover** (A). A linear correction C applied to x leaves $x - Cx$. That vector is zero if and only if $Cx = x$. A leftover is not a decision and not an origin story.

This package does not invent a seventh calculus that absorbs those six. It records how a finite evaluator may call them in order, and it records the one sentence the six jobs already separate: a leftover class is not standing.

The theory is refuted by any of: a leftover class treated as standing; a commuting square treated, by itself, as target standing; a unique glue of incompatible overlaps treated as a grant; a coarse look treated as recovering the forgotten bit; a closed leftover routed to the standing branch of the finite evaluator below; or a bicategory of all comparison systems claimed as a theorem of this note.

Notation

- FOC, FDA, MUR, OR, CS, and A are the six companion papers cited in the references. Q and the CS–Q bridge are later notes; they are not jobs of this spine.
- A *finite presentation* is a named finite carrier V together with a symmetric, zero-on-diagonal table of nonnegative differences.
- A *comparison span* from P_1 to P_2 is an intermediate carrier M with maps $M \to V_1$ and $M \to V_2$, together with a nonnegative *span cost*.
- A *trace* is a span plus a step count and a nonnegative *work cost*. Forgetting the work leaves the span.
- $\Phi_1([x])$ is the cheapest first-stage leftover of a defect x after licensed linear repairs, as in FOC. $\Gamma_2([d_1 x])$ is the cheapest next-stage leftover. Those numbers live in FOC, not here.
- Standing, as in FDA, is a dependent pair: a terminal actually produced, and seven derived halt guards. This note does not re-prove that pair.
- A *leftover class* is the class of a vector in the cokernel of a projection, as in A. It is a class in a quotient, not a standing record.

The FDA guard identifiers are g_1 `accepted`, g_2 `deterministic`, g_3 `zero_residual_mass`, g_4 `skeleton_preservin g`, g_5 `exhaustive`, g_6 `no_pending_lift`, and g_7 `no_refusal`. These names refer to `coda_fda_core.derive_guards` in the separate FDA package. They are requirements on evidence for the selected terminal; the app firewall uses a different checklist.

A **finite pipeline record** in this package is a declared tuple of flags and leftover numbers: whether a candidate list is nonempty, whether a coarse look matches a forgotten fine look, whether declared

overlaps agree, the two leftover numbers, and whether a standing licence flag is set. The evaluator below reads that tuple. It does not construct the six companion theorems.

1. Six jobs, cited not re-proved

Each job has a home paper. This section names the job and points. It does not replay the proofs.

Obstruction — FOC

A defect is a vector. Licensed repairs add vectors from a declared image $\operatorname{im} d_0$. The leftover $\Phi_1([x])$ is the cheapest weighted ℓ^1 distance from x to that image. A dual vector that annihilates the whole repair subspace certifies a positive leftover. Three labels exhaust a well-posed input: exact repair, coherent residual, or nonclosed defect. The labels classify the vector. They do not choose an action. See FOC Theorems FOC-T1–T4.

Standing — FDA

A proposal is not a decision. Standing is inhabited only when a five-field packet exists, a four-stage pipeline returns a left injection $\mathsf{inl}(t)$, and seven halt guards derived from evidence hold of t. A confidence score is not that pair. One of the seven guards may read leftover mass from the FOC solver; that solver measures leftover. See FDA Definitions FDA-D1–D2 and Theorems FDA-T1–T7.

Transport — MUR

A transport is a commuting square: $q(F(x)) = \bar{F}(p(x))$. Two such squares paste. If $p(x) = p(y)$, the images still collide. A later function of the observation cannot split a collision. The square is a certificate that the move is compatible with looking. It is not, by itself, a standing record in the target. See MUR Theorems MUR-T1–T4.

Comparison — OR

Pairs (a, b) and (b', c) glue to a unique triple if and only if $b = b'$. An empty support sieve is a hard refusal. Four square labels — strict commute, exact repair, closed leftover, open leftover — are diagnoses of a defect, not grants. See OR Theorems OR-T1–T4.

Shadow — CS

A fine look forgotten along forget is a coarse look. Fibre constancy on the coarse fibres implies fibre constancy on the fine fibres. Four Boolean pairs collapse onto two first coordinates; the second coordinate does not factor through the first. A written limit is a name, not an answer. See CS Theorems CS-T1–T5.

Leftover — A

$\operatorname{residual}(C, x) := x - Cx$. That leftover is zero if and only if $Cx = x$. If C is a projection, the leftover lies in $\ker C$, and leftover of leftover is leftover. A linear map that commutes with two corrections carries leftovers to leftovers. Two vectors have the same leftover if and only if they differ by a vector in $\operatorname{im} C$. A leftover is not a decision and not an origin story. See A Theorems A-T1–T7 and Proposition A-P1.

Those six jobs remain six jobs. Composing them in a finite evaluator does not identify leftover with standing, glue with a grant, or a shadow with the original.

2. Local records: presentations, spans, traces

What this package adds is small. It records finite presentations, comparison spans, traces, and an evaluator on declared finite records. The Lean namespace is `CodaGtla`. The Lean carriers are types with decidable equality and a difference table valued in $\mathbb{N}$.

Definition GTLA-D1 — finite presentation

The Lean presentation record is a type V of carrier points, with decidable equality, and a map $\mathrm{diff} : V \to V \to \mathbb{N}$ such that $\mathrm{diff}(x, x) = 0$ for every x. The Lean record is `CodaGtla.Presentation`.

Finiteness is an explicit Python restriction, not a field of the Lean record. The Python companion also asks the table to be symmetric and the carrier to be inhabited. Those extra checks are implementation contracts, not extra Lean axioms.

Definition GTLA-D2 — comparison span

A comparison span from P_1 to P_2 is an intermediate type M, maps $M \to P_1.V$ and $M \to P_2.V$, and a span cost in $\mathbb{N}$. The Lean record is `CodaGtla.ComparisonSpan`.

The *diagonal span* on P takes $M = P.V$ and both maps equal to the identity, with cost 0. Composition of spans $S_1 : P_1 \to P_2$ and $S_2 : P_2 \to P_3$ is the pullback set of pairs (m_1, m_2) with matching middle point, with cost the sum of the two costs.

Theorem GTLA-T1 — diagonal cost is zero

The diagonal span has cost 0.

Proof. By definition of `diagSpan`. This is Lean `CodaGtla.diagSpan_cost_zero`. □

Theorem GTLA-T2 — composite cost is subadditive

The cost of a composite span is at most the sum of the costs.

Proof. In this record the composite cost *is* the sum, so the inequality is reflexivity of $\leq$ on $\mathbb{N}$. This is Lean `CodaGtla.compSpan_cost_subadditive`. □

The inequality is recorded because a coarser cost, or a cost that dropped duplicates in the pullback, would still have to obey it. This package does not prove a bicategory of all comparison systems, and it does not identify these spans with the gluing theorem of OR. OR-T1 is about pairs in $A \times B$ and $B \times C$. The span record is a finite diagram with a cost. They are neighbours, not the same theorem.

Definition GTLA-D3 — trace

A trace from P_1 to P_2 is a step count, a work cost in $\mathbb{N}$, and an underlying span. The Lean record is `CodaGtla.Trace`. The identity trace has zero steps, zero work, and the diagonal span. Composition adds steps, adds work, and composes the underlying spans. *Erasure* forgets steps and work and returns the span.

Theorem GTLA-T3 — identity work is zero

The identity trace has work cost 0.

Proof. By definition of `idTrace`. This is Lean `CodaGtla.idTrace_work_zero`. □

Theorem GTLA-T4 — erasure preserves identity

Erasing the identity trace yields the diagonal span.

Proof. By definition. This is Lean `CodaGtla.eraseTrace_id`. □

Theorem GTLA-T5 — erasure preserves composition

Erasing a composite trace yields the composite of the erased spans.

Proof. By definition. This is Lean `CodaGtla.eraseTrace_comp`. □

So work can be positive while the underlying span is a round-trip of cost zero. That is a two-level accounting on these records. It is not a theorem that every groupoid embeds, and it is not a claim that computational effort is a physical observable. The Python companion exhibits the accounting on the finite groups $\mathbb{Z}/2\mathbb{Z}$ and $\mathbb{Z}/3\mathbb{Z}$ as worked examples.

3. A leftover class is not standing

The six jobs already separate leftover from standing. This package records the separation on a finite evaluator, so a reader can see it fail in the way the tests are written to fail.

Definition GTLA-D4 — three terminal kinds

A terminal of the finite evaluator is exactly one of:

- a standing branch, labelled `authorized` in the Lean inductive;
- a closed-leftover branch, labelled `closedResidual`;
- a refusal branch, labelled `refused`.

The three predicates `IsAuth`, `IsAnomalon`, and `IsRefused` read those constructors. The names in the code are historical. The residual constructor stores an identifier and a natural-number distance; it does not itself construct a quotient class from A. The authorized constructor likewise does not construct an FDA standing record.

Definition GTLA-D5 — finite pipeline record

A pipeline input is a declared tuple

$$(\texttt{sieveNonEmpty},\ \texttt{refinementConsistent},\ \texttt{gluingSucceeded},\ \Phi_1,\ \Gamma_2,\ \texttt{licenseGranted}).$$

The evaluator `evaluatePipeline` reads the tuple in order: empty candidate list refuses; inconsistent refinement or failed glue refuses; positive next-stage leftover refuses; zero first-stage leftover with the licence flag set takes the standing branch; zero first-stage leftover with the licence flag unset refuses; a positive first-stage leftover with zero next-stage leftover takes the leftover branch.

Those flags are inputs. The evaluator does not prove FOC duality, FDA standing soundness, OR gluing uniqueness, or CS fibre constancy. It copies declared answers.

Theorem GTLA-T6 — empty sieve is a refusal

If `sieveNonEmpty` is false, the evaluator returns a refusal.

Proof. The first branch of `evaluatePipeline` is that check. This is Lean `CodaGtla.pipeline_empty_sieve_refused`. □

That is the OR empty-sieve job, cited not re-proved: an empty support sieve is a hard refusal (OR). Here the sieve is a Boolean on a finite record.

Theorem GTLA-T7 — a leftover class is not standing

If the candidate list is nonempty, refinement and glue flags are true, the next-stage leftover is zero, and the first-stage leftover is positive, the evaluator returns a closed leftover, not a standing branch.

Proof. After the refusal checks, the remaining case with $\Phi_1 > 0$ is `closedResidual`. This is Lean `CodaGtla.pipeline_closed_residual_anomalon`. □

That is the A job, cited not re-proved: leftover of a projection is a vector in a kernel, and a leftover is not a decision (A-T1, A-P1). The finite evaluator records the same refusal to treat a positive leftover as a yes.

Theorem GTLA-T8 — the three kinds are pairwise disjoint

No terminal is both a standing branch and a leftover class, both a standing branch and a refusal, or both a leftover class and a refusal.

Proof. Case analysis on the inductive. This is Lean `CodaGtla.auth_disjoint_anomalon`, `auth_disjoint_refused`, and `anomalon_disjoint_refused`. □

Disjoint constructors are not a conservation law for a tower. They are the statement that this inductive has three different constructors.

Theorem GTLA-T9 — zero leftover without a licence is not standing

There is a pipeline record with $\Phi_1 = 0$, licence flag false, whose output is a refusal and not a standing branch.

Proof. Take a nonempty sieve, consistent refinement, successful glue, both leftover numbers zero, and `licenseGranted=false`. The evaluator hits the unlicensed zero-leftover refusal. This is Lean `CodaGtla.non_promotion_firewall`. □

Zero leftover means, in FOC, that the defect already lies in the repair image. In A, leftover zero means $Cx = x$. Neither sentence is a standing record. FDA standing still needs a packet, a pipeline path, and seven guards. The licence flag in this finite record is a declared stand-in for that pair, not a proof of FDA-T7.

Theorem GTLA-T10 — the same leftover numbers can disagree on standing

There are two pipeline records with equal Φ_1 and equal Γ_2 such that one lands in the standing branch and the other does not.

Proof. Take both leftover numbers zero, both sieves nonempty, both refinement and glue flags true. Set the licence flag true on the first record and false on the second. This is Lean `CodaGtla.observation_collision_distinct_authority`. □

FDA already exhibits an observation collision: two states can look the same and disagree on standing (FDA-04, cited). CS already exhibits four dots that share a first coordinate and disagree on the second (CS-T3, cited). The finite evaluator records the same pattern on leftover numbers: Φ_1 and Γ_2 do not determine the standing branch.

A licensed zero-leftover record does take the standing branch of this evaluator (Lean `pipeline_licensed_exact_authorized`). That is a clause of the finite `if`. It is not FDA standing, and it is not a theorem that zero leftover creates a grant. The interesting clauses are Theorems GTLA-T7 and GTLA-T9: leftover is not a yes, and zero leftover is not a yes either.

4. When the layers disagree

The six jobs can agree or disagree. Disagreement is data, not a bug to be averaged away. OR already forbids averaging conflicting detectors. This section lists the finite disagreements the companion actually runs.

Empty list versus leftover. An empty candidate list refuses before any leftover is computed. The obstruction desk never sits. That is Theorem GTLA-T6.

Failed glue versus leftover. If declared overlaps disagree, the evaluator refuses. The leftover numbers are not consulted. Comparison failed; obstruction is not asked to paper over it.

Open leftover versus closed leftover. A positive next-stage leftover is a nonclosed defect in FOC. The evaluator refuses. A positive first-stage leftover with zero next-stage leftover is a coherent FOC residual. The evaluator records that diagnostic label. Its relation to an A projection requires the hypotheses stated in Section 4.3. Those are different constructors (Theorem GTLA-T8).

Closed leftover versus standing. Theorem GTLA-T7. The leftover desk returns a class. The standing desk is not thereby inhabited.

Zero leftover versus standing. Theorem GTLA-T9. The obstruction desk returns zero. The standing desk still needs its own pair.

Same leftover, different standing. Theorem GTLA-T10. Two records can share Φ_1 and Γ_2 and disagree on the licence flag. Looking at the leftover numbers cannot tell them apart.

The Python companion runs these cases as exact rational matrices. One worked triple:

1. Defect $(1, -1)$ with repair map $d_0 = (1, -1)^\top$ and next map $d_1 = (1, 1)$, licence flag true: first-stage leftover 0, standing branch of the finite evaluator.
2. Defect $(1, 1)$ with the zero repair map and zero next map, licence flag false: first-stage leftover 2, leftover branch.
3. Empty candidate list: refusal, no leftover computed.

The first case is not a proof that leftover zero is standing. It is the evaluator reading a declared licence flag. Drop that flag and the same defect is a refusal (Theorem GTLA-T9). Treat case 2 as a yes, and the tests fail.

4.3 Coalesced Synthesis of the Six Foundational Invariant Boundaries

This monograph integrates the foundational results established across the Finite Obstruction Calculus series (Papers 004, 005, 027, 028, 029, 030):

1. **Role Non-Impersonation Program (Paper 004):** Proposals are not authority ($p \in \mathcal{P} \not\Rightarrow \mathsf{Auth}(p)$). The six governance tower positions ($\partial, \mathbb{A}, \chi, \mathfrak{O}, \kappa, \Lambda\Omega$) form non-identifiable roles; internal evaluators cannot serve as terminal authority for their own outputs (rubber-stamp prohibition).
2. **The Authority Gap & Exactness Fraud (Paper 005):** A vanishing quotient defect does not imply semantic correctness ($\Phi_1 = 0 \not\Rightarrow$ Truth). Observers erase defects rather than repairing them (Lemma O1). An exact zero defect without a matching declared licence routes to $\mathbb{F}_{\text{term}}$ (`REFUSED`) in this package, by GTLA-T9.

3. **Coproduct Separation Theorem (Paper 027):** The terminal certificate type splits as a coproduct $\mathbb{T} = \mathbb{T}_{\mathsf{auth}} \oplus (\mathbb{T}_{\mathrm{res}} + \mathbb{F}_{\mathrm{res}})$. No residual or diagnostic outcome inhabits the authority summand in the declared sum type. *The proposed absence of natural transformations from residual data to authority (OPEN). (WITHDRAWN 2026-09-28: the inference from disjoint coproduct constructors to an empty set of natural transformations is false in general; countermodel: two constant singleton functors admit a unique natural transformation while their coproduct injections remain distinct.)* Constructor disjointness (Theorems GTLA-T8 and FDA-C1) is retained; promotion restrictions from residual outcomes to standing are an interface policy of the declared system. A stronger categorical no-map statement would require additional hypotheses not supplied here.
4. **Finite Authority & Non-Terminal Computation (Paper 028):** Reasoning authority is not generated; it is terminalized. Universal symbol manipulation is not universal authority. Exhaustion of a declared time or step budget, or empty candidate support sieves, is required by the external policy to fail closed into $\mathbb{F}_{\mathrm{term}}$ (`REFUSED`). The finite GTLA evaluator has no timer or unbounded-loop implementation.
5. **The Anomalon Boundary Invariant (Paper 029):** The root ontological boundary is $\mathrm{Real}_0 =$ Primitive Difference $\cup$ Structure. *Non-zero coherent obstructions ($\Phi_1 > 0, \Gamma_2 = 0$) are irreducible in the cokernel $V/\operatorname{im}(C)$ and must be locked into the Anomalon register without smoothing. (WITHDRAWN 2026-09-23: C and d_0 were unrelated.)* With C idempotent and $\operatorname{im} C = \operatorname{im} d_0$, a class with $\Phi_1 > 0$ has non-zero leftover $x - Cx$, and no repair from $\operatorname{im} d_0$ changes it (CODA-A Theorem A-T9, restated 2026-09-23). Retaining such a class in a register is a declared policy.
6. **Verification Limits & Authority Typing (Paper 030):** Verifiers face the elaboration-is-not-auditing obstruction (C4 irreducible trust root). Authority is strictly typed as `ContractAuthority` and `VerifierScopedAuthority`; `SemanticAuthority` is UNAVAILABLE without an external ground-truth oracle.

5. Computational companion

The production module `code/coda_gtla_core.py` stores finite presentations and spans, adds trace work, and runs the finite evaluator on exact rational matrices. Arithmetic is exact rational; floats and booleans in scalar position raise `TypeError`. The FOC leftover numbers, when the glue flag is true, are delegated to the shipped ℓ^1 quotient solver of the obstruction companion. Substituting an ℓ^2 leftover, granting the standing branch on leftover zero without the licence flag, or routing a closed leftover to the standing branch, is a fault the tests are written to kill.

Lean files in `lean4/`:

- `D01D02Records.lean` — presentations, spans, traces, the three terminal constructors;
- `T01SpanPullback.lean` — Theorems GTLA-T1–T5;
- `T02MasterNormalForm.lean` — the finite evaluator and Theorems GTLA-T6–T7;
- `T03AuthorityFirewall.lean` — Theorems GTLA-T8–T10;
- `Algebra.lean` — wraps reconstruction leftover identities from A, so this package can *cite* them; it does not re-prove them;
- `Core.lean` — re-exports the files above.

There are no `sorry` stubs in those files. The local theorems use the ordinary Lean kernel. Bicategories, H3'' glue of overlapping shadows, and a general linear-programming duality theorem are not proved here.

Adversarial tests include: identity span cost zero; empty sieve refusal; closed leftover not standing; zero leftover without a licence flag not standing; two records with equal leftover numbers and unequal standing branches; float rejection.

5.1 Finite evidence boundaries and routing

Profiles must have equal lengths. With a structured cover, gluing checks every shared coordinate of nonempty supplied charts, including intersections omitted from the optional overlap-flag table. The returned global domain and section restrict to all local sections. Without a cover, tuple indices are the declared common domain. A caller-supplied flag cannot certify inconsistent values.

Both quotient distances are computed exactly whenever the FOC stage runs. A skipped stage records absent distances. Terminalization rechecks its stage records against the input. Full-record digests bind the input matrices, profiles, sections, witnesses, token fields and outcomes; they do not authenticate those records. `AuthorityToken` is a declared model licence, with subject and scope checks. Its validity field and opaque signature label are assumptions; no external signature verification is implemented.

First applicable condition	Local output
Empty sieve, inconsistent refinement, failed gluing, or mismatched stage evidence	Refusal; skipped distances absent
Successful earlier stages and $\Gamma_2 > 0$	Refusal with computed distances
Successful earlier stages, $\Phi_1 = \Gamma_2 = 0$, matching declared licence	Authorized constructor in the model
Successful earlier stages, $\Phi_1 = \Gamma_2 = 0$, no matching declared licence	Refusal
Successful earlier stages, $\Phi_1 > 0$, $\Gamma_2 = 0$	Closed residual with exact dual certificate, independently of licence

The historical `execute_two_rule_repair_loop` name now returns a repair or positive-residual report. It supplies no extended carrier and no enumeration of tied minimizers; `ties_retained=None` records that unevaluated obligation.

5.2 A bounded two-row cycle quotient

The Python bicomplex has $0 \le p \le 2$ and $0 \le q \le 1$, exact rational matrices, horizontal square-zero laws, and anticommuting horizontal/vertical maps. For an $\omega \in C^{0,1}$ with $d_h\omega$ in the image of d_v, solve $d_v\eta = -d_h\omega$. The target of the resulting class is

$$E_2^{2,0} = \ker(d_v^{2,0})/d_h^{1,0}(\ker(d_v^{1,0})).$$

Anticommutation and $d_h^2 = 0$ put $d_h\eta$ in the numerator. Two choices of η differ by a vertical cycle, so their outputs differ by a displayed boundary. Replacing ω by a vertical boundary change changes η by a horizontal image and does not change this class. This justifies the bounded cycle quotient used by `certify_d2_quotient_bounds`. Inputs and optional fluxes outside the required kernels are rejected.

By contrast, $[d_h\eta; 0] = D_1[\eta; \omega]$ is always a total-complex boundary. Its distance modulo $\operatorname{im} D_1$ is zero and cannot measure the E2 class. The earlier positive triangle example used an invalid flux and is withdrawn. The canonical triangle has a zero E2 target quotient.

A genuine finite nonzero example takes all six component spaces to be $\mathbb{Q}$, $d_h^{0,1} = d_h^{1,0} = d_v^{1,0} = 1$, and every other map zero. All four bicomplex identities hold. For $\omega = 1$, $\eta = -1$, the E2 boundary space is zero, and the class -1 has exact quotient distance 1. The test checks the primal and dual witnesses as well as the total-boundary identity. This is a bounded two-row example, not a general spectral-sequence construction. Counting faces through degree two gives no proof that truncation preserves cohomology or that LP enumeration is polynomial-time.

6. Limitations

The local theorems are equalities and case splits on declared finite records. They hold for those records. They are not theorems of FOC, FDA, MUR, OR, CS, or A. Those papers keep their own theorems.

This note does not prove a bicategory of comparison systems, a simplicial duality, an origin of leftover, or a standing record from a leftover class. Q and the CS–Q bridge are later notes: metric completion and Lipschitz access are not jobs of this spine.

A leftover class is a class in a quotient. Standing is a recorded pair. Theorem GTLA-T7 is the whole claim of the spine: those are different jobs.

References

1. J. H. Carroll, *The Finite Obstruction Calculus: Exact quotient diagnostics, dual witnesses, two-stage coherence, and transport (CODA-FOC)*, 2026.
2. J. H. Carroll, *Finite Descent Authority: Packets, Pipelines, and Seven Halt Guards: A proposal is not a decision (CODA-FDA)*, 2026.
3. J. H. Carroll, *Meta & Universal Re-Presentation: Commuting Squares, Composition, and Irreversible Forgetting: A commuting square is not target standing (CODA-MUR)*, 2026.
4. J. H. Carroll, *Omniversal Re-Presentation: Re-Presentations Transport Rules (CODA-OR)*, 2026.
5. J. H. Carroll, *The Continuum Shadow: Continuum Re-presentations that Descend to the Finite Calculus (CODA-CS)*, 2026.
6. J. H. Carroll, *The Continuum Shadow to Continuum Bridge: Finite completion and enclosure (CODA-CS-Q)*, 2026.
7. J. H. Carroll, *The Continuum: Continuum Representations via Completion and Enclosure (CODA-Q)*, 2026.
8. J. H. Carroll, *The Anomalon: Residuals, Kernels, and Cokernels (CODA-A)*, 2026.
9. S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998. https://doi.org/10.1007/978-1-4757-4721-8
10. S. Axler, *Linear Algebra Done Right*, 3rd ed., Springer, 2015. https://doi.org/10.1007/978-3-319-11080-6